

RELATÓRIO

Cadastro de Clientes em Java

Curso: Desenvolvimento Web Full Stack

Disciplina: Programação Back-end com Java

Aluno(a): Vivian Maria Avelino Coelho

Repositório GIT : <https://github.com/vivianavelino/cadastroClientejava.git>

1. Título da Prática

Desenvolvimento de um sistema de cadastro de clientes em Java utilizando orientação a objetos e persistência em arquivos binários.

2. Objetivo da Prática

foi desenvolver um sistema simples de cadastro de clientes em modo texto. O programa trabalha com pessoas físicas e pessoas jurídicas e permite incluir, alterar, excluir, consultar e listar registros. Também foi implementada a gravação e a recuperação dos dados em arquivos binários.

Durante o desenvolvimento foram utilizados conceitos estudados na disciplina, como herança, polimorfismo, encapsulamento, ArrayList, Serializable, Scanner e tratamento de exceções.

3. Organização do Projeto

O projeto foi dividido em dois pacotes. O pacote model contém as classes das entidades e os repositórios. O pacote cadastropoo contém a classe principal CadastroPOO, responsável pelo menu e pela interação com o usuário.

```
trabalhojava
|-- cadastropoo
|   |-- CadastroPOO.java
|-- model
|   |-- Pessoa.java
|   |-- PessoaFisica.java
|   |-- PessoaJuridica.java
|   |-- PessoaFisicaRepo.java
|   |-- PessoaJuridicaRepo.java
|-- clientes.fisica.bin
|-- clientes.juridica.bin
```

4. Códigos Desenvolvidos

Pessoa.java

```
package model;

import java.io.Serializable;
```

```

public class Pessoa implements Serializable {

    private int id;
    private String nome;

    public Pessoa() {
    }

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public void exibir() {
        System.out.println("ID: " + id);
        System.out.println("Nome: " + nome);
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}

```

PessoaFisica.java

```

package model;

import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {

    private String cpf;
    private int idade;

    public PessoaFisica() {
    }

    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }

    @Override
    public void exibir() {
        System.out.println("ID: " + getId());
        System.out.println("Nome: " + getNome());
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }

    public String getCpf() {
        return cpf;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
}

```

```
}  
}
```

PessoaJuridica.java

```
package model;  
  
import java.io.Serializable;  
  
public class PessoaJuridica extends Pessoa implements Serializable {  
  
    private String cnpj;  
  
    public PessoaJuridica() {  
    }  
  
    public PessoaJuridica(int id, String nome, String cnpj) {  
        super(id, nome);  
        this.cnpj = cnpj;  
    }  
  
    @Override  
    public void exibir() {  
        System.out.println("ID: " + getId());  
        System.out.println("Nome: " + getNome());  
        System.out.println("CNPJ: " + cnpj);  
    }  
  
    public String getCnpj() {  
        return cnpj;  
    }  
  
    public void setCnpj(String cnpj) {  
        this.cnpj = cnpj;  
    }  
}
```

PessoaFisicaRepo.java

```
package model;  
  
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.ObjectInputStream;  
import java.io.ObjectOutputStream;  
import java.util.ArrayList;  
  
public class PessoaFisicaRepo {  
  
    private ArrayList<PessoaFisica> pessoasFisicas;  
  
    public PessoaFisicaRepo() {  
        pessoasFisicas = new ArrayList<>();  
    }  
  
    public void inserir(PessoaFisica pessoa) {  
        pessoasFisicas.add(pessoa);  
    }  
  
    public void alterar(PessoaFisica pessoa) {  
        for (int i = 0; i < pessoasFisicas.size(); i++) {  
            if (pessoasFisicas.get(i).getId() == pessoa.getId()) {  
                pessoasFisicas.set(i, pessoa);  
                break;  
            }  
        }  
    }  
  
    public void excluir(int id) {  
        for (int i = 0; i < pessoasFisicas.size(); i++) {  
            if (pessoasFisicas.get(i).getId() == id) {  
                pessoasFisicas.remove(i);  
                break;  
            }  
        }  
    }  
}
```

```

public PessoaFisica obter(int id) {
    for (PessoaFisica pessoa : pessoasFisicas) {
        if (pessoa.getId() == id) {
            return pessoa;
        }
    }
    return null;
}

public ArrayList<PessoaFisica> obterTodos() {
    return pessoasFisicas;
}

public void persistir(String nomeArquivo) throws Exception {
    FileOutputStream arquivo = new FileOutputStream(nomeArquivo);
    ObjectOutputStream objeto = new ObjectOutputStream(arquivo);
    objeto.writeObject(pessoasFisicas);
    objeto.close();
    arquivo.close();
}

public void recuperar(String nomeArquivo) throws Exception {
    FileInputStream arquivo = new FileInputStream(nomeArquivo);
    ObjectInputStream objeto = new ObjectInputStream(arquivo);
    pessoasFisicas = (ArrayList<PessoaFisica>) objeto.readObject();
    objeto.close();
    arquivo.close();
}
}

```

PessoaJuridicaRepo.java

```

package model;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

public class PessoaJuridicaRepo {

    private ArrayList<PessoaJuridica> pessoasJuridicas;

    public PessoaJuridicaRepo() {
        pessoasJuridicas = new ArrayList<>();
    }

    public void inserir(PessoaJuridica pessoa) {
        pessoasJuridicas.add(pessoa);
    }

    public void alterar(PessoaJuridica pessoa) {
        for (int i = 0; i < pessoasJuridicas.size(); i++) {
            if (pessoasJuridicas.get(i).getId() == pessoa.getId()) {
                pessoasJuridicas.set(i, pessoa);
                break;
            }
        }
    }

    public void excluir(int id) {
        for (int i = 0; i < pessoasJuridicas.size(); i++) {
            if (pessoasJuridicas.get(i).getId() == id) {
                pessoasJuridicas.remove(i);
                break;
            }
        }
    }

    public PessoaJuridica obter(int id) {
        for (PessoaJuridica pessoa : pessoasJuridicas) {
            if (pessoa.getId() == id) {
                return pessoa;
            }
        }
        return null;
    }
}

```

```

    }

    public ArrayList<PessoaJuridica> obterTodos() {
        return pessoasJuridicas;
    }

    public void persistir(String nomeArquivo) throws Exception {
        FileOutputStream arquivo = new FileOutputStream(nomeArquivo);
        ObjectOutputStream objeto = new ObjectOutputStream(arquivo);
        objeto.writeObject(pessoasJuridicas);
        objeto.close();
        arquivo.close();
    }

    public void recuperar(String nomeArquivo) throws Exception {
        FileInputStream arquivo = new FileInputStream(nomeArquivo);
        ObjectInputStream objeto = new ObjectInputStream(arquivo);
        pessoasJuridicas = (ArrayList<PessoaJuridica>) objeto.readObject();
        objeto.close();
        arquivo.close();
    }
}

```

CadastroPOO.java

```

package cadastratpoo;

import java.util.Scanner;
import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

public class CadastroPOO {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);
        PessoaFisicaRepo repoFisica = new PessoaFisicaRepo();
        PessoaJuridicaRepo repoJuridica = new PessoaJuridicaRepo();

        int opcao = -1;

        while (opcao != 0) {

            System.out.println();
            System.out.println("=====");
            System.out.println("      CADASTRO DE CLIENTES");
            System.out.println("=====");
            System.out.println("1 - Incluir");
            System.out.println("2 - Alterar");
            System.out.println("3 - Excluir");
            System.out.println("4 - Exibir pelo ID");
            System.out.println("5 - Exibir todos");
            System.out.println("6 - Salvar dados");
            System.out.println("7 - Recuperar dados");
            System.out.println("0 - Sair");
            System.out.print("Escolha uma opcao: ");

            opcao = Integer.parseInt(teclado.nextLine());

            switch (opcao) {

                case 1:
                    System.out.println();
                    System.out.println("1 - Pessoa Fisica");
                    System.out.println("2 - Pessoa Juridica");
                    System.out.print("Escolha o tipo: ");

                    int tipoInclusao = Integer.parseInt(teclado.nextLine());

                    if (tipoInclusao == 1) {
                        System.out.print("ID: ");
                        int id = Integer.parseInt(teclado.nextLine());
                        System.out.print("Nome: ");
                        String nome = teclado.nextLine();
                        System.out.print("CPF: ");
                        String cpf = teclado.nextLine();
                    }
                }
            }
        }
    }
}

```

```

        System.out.print("Idade: ");
        int idade = Integer.parseInt(teclado.nextLine());

        PessoaFisica pessoa = new PessoaFisica(id, nome, cpf, idade);
        repoFisica.inserir(pessoa);
        System.out.println("Pessoa fisica cadastrada.");

    } else if (tipoInclusao == 2) {
        System.out.print("ID: ");
        int id = Integer.parseInt(teclado.nextLine());
        System.out.print("Nome: ");
        String nome = teclado.nextLine();
        System.out.print("CNPJ: ");
        String cnpj = teclado.nextLine();

        PessoaJuridica pessoa = new PessoaJuridica(id, nome, cnpj);
        repoJuridica.inserir(pessoa);
        System.out.println("Pessoa juridica cadastrada.");
    } else {
        System.out.println("Tipo invalido.");
    }
    break;

case 2:
    System.out.println();
    System.out.println("1 - Pessoa Fisica");
    System.out.println("2 - Pessoa Juridica");
    System.out.print("Escolha o tipo: ");

    int tipoAlteracao = Integer.parseInt(teclado.nextLine());
    System.out.print("Digite o ID: ");
    int idAlteracao = Integer.parseInt(teclado.nextLine());

    if (tipoAlteracao == 1) {
        PessoaFisica pessoa = repoFisica.obter(idAlteracao);

        if (pessoa != null) {
            System.out.println();
            System.out.println("Dados atuais:");
            pessoa.exibir();

            System.out.println();
            System.out.println("Digite os novos dados:");
            System.out.print("Nome: ");
            String nome = teclado.nextLine();
            System.out.print("CPF: ");
            String cpf = teclado.nextLine();
            System.out.print("Idade: ");
            int idade = Integer.parseInt(teclado.nextLine());

            PessoaFisica novaPessoa =
                new PessoaFisica(idAlteracao, nome, cpf, idade);
            repoFisica.alterar(novaPessoa);
            System.out.println("Cadastro alterado.");
        } else {
            System.out.println("Pessoa nao encontrada.");
        }
    }

    } else if (tipoAlteracao == 2) {
        PessoaJuridica pessoa = repoJuridica.obter(idAlteracao);

        if (pessoa != null) {
            System.out.println();
            System.out.println("Dados atuais:");
            pessoa.exibir();

            System.out.println();
            System.out.println("Digite os novos dados:");
            System.out.print("Nome: ");
            String nome = teclado.nextLine();
            System.out.print("CNPJ: ");
            String cnpj = teclado.nextLine();

            PessoaJuridica novaPessoa =
                new PessoaJuridica(idAlteracao, nome, cnpj);
            repoJuridica.alterar(novaPessoa);
            System.out.println("Cadastro alterado.");
        } else {
            System.out.println("Pessoa nao encontrada.");
        }
    }
}

```

```

    } else {
        System.out.println("Tipo invalido.");
    }
    break;

case 3:
    System.out.println();
    System.out.println("1 - Pessoa Fisica");
    System.out.println("2 - Pessoa Juridica");
    System.out.print("Escolha o tipo: ");

    int tipoExclusao = Integer.parseInt(teclado.nextLine());
    System.out.print("Digite o ID: ");
    int idExclusao = Integer.parseInt(teclado.nextLine());

    if (tipoExclusao == 1) {
        PessoaFisica pessoa = repoFisica.obter(idExclusao);

        if (pessoa != null) {
            repoFisica.excluir(idExclusao);
            System.out.println("Pessoa fisica excluida.");
        } else {
            System.out.println("Pessoa nao encontrada.");
        }
    }

    } else if (tipoExclusao == 2) {
        PessoaJuridica pessoa = repoJuridica.obter(idExclusao);

        if (pessoa != null) {
            repoJuridica.excluir(idExclusao);
            System.out.println("Pessoa juridica excluida.");
        } else {
            System.out.println("Pessoa nao encontrada.");
        }
    }
    } else {
        System.out.println("Tipo invalido.");
    }
    break;

case 4:
    System.out.println();
    System.out.println("1 - Pessoa Fisica");
    System.out.println("2 - Pessoa Juridica");
    System.out.print("Escolha o tipo: ");

    int tipoObter = Integer.parseInt(teclado.nextLine());
    System.out.print("Digite o ID: ");
    int idObter = Integer.parseInt(teclado.nextLine());

    if (tipoObter == 1) {
        PessoaFisica pessoa = repoFisica.obter(idObter);

        if (pessoa != null) {
            System.out.println();
            pessoa.exibir();
        } else {
            System.out.println("Pessoa nao encontrada.");
        }
    }

    } else if (tipoObter == 2) {
        PessoaJuridica pessoa = repoJuridica.obter(idObter);

        if (pessoa != null) {
            System.out.println();
            pessoa.exibir();
        } else {
            System.out.println("Pessoa nao encontrada.");
        }
    }
    } else {
        System.out.println("Tipo invalido.");
    }
    break;

case 5:
    System.out.println();
    System.out.println("1 - Pessoa Fisica");
    System.out.println("2 - Pessoa Juridica");
    System.out.print("Escolha o tipo: ");

    int tipoTodos = Integer.parseInt(teclado.nextLine());

```

```

        if (tipoTodos == 1) {
            System.out.println();
            System.out.println("PESSOAS FISICAS");

            for (PessoaFisica pessoa : repoFisica.obterTodos()) {
                pessoa.exibir();
                System.out.println();
            }

        } else if (tipoTodos == 2) {
            System.out.println();
            System.out.println("PESSOAS JURIDICAS");

            for (PessoaJuridica pessoa : repoJuridica.obterTodos()) {
                pessoa.exibir();
                System.out.println();
            }

        } else {
            System.out.println("Tipo invalido.");
        }
        break;

    case 6:
        System.out.print("Digite o prefixo dos arquivos: ");
        String prefixoSalvar = teclado.nextLine();

        try {
            repoFisica.persistir(prefixoSalvar + ".fisica.bin");
            repoJuridica.persistir(prefixoSalvar + ".juridica.bin");
            System.out.println("Dados salvos com sucesso.");
        } catch (Exception e) {
            System.out.println("Erro ao salvar os dados.");
            System.out.println("Detalhes: " + e.getMessage());
        }
        break;

    case 7:
        System.out.print("Digite o prefixo dos arquivos: ");
        String prefixoRecuperar = teclado.nextLine();

        try {
            repoFisica.recuperar(prefixoRecuperar + ".fisica.bin");
            repoJuridica.recuperar(prefixoRecuperar + ".juridica.bin");
            System.out.println("Dados recuperados com sucesso.");
        } catch (Exception e) {
            System.out.println("Erro ao recuperar os dados.");
            System.out.println("Detalhes: " + e.getMessage());
        }
        break;

    case 0:
        System.out.println();
        System.out.println("Programa finalizado.");
        break;

    default:
        System.out.println("Opcao invalida.");
    }
}

teclado.close();
}
}

```

5. Resultados da Execução

Após a implementação, as funções do cadastro foram testadas no terminal. Foram realizados testes de inclusão de pessoa física e jurídica, listagem, busca por ID, alteração, exclusão, salvamento e recuperação dos dados.

```
PS C:\Users\vivia\OneDrive\Desktop\trabalhojava> c:: cd 'c:\Users\vivia\OneDrive\Desktop\trabalhojava'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivia\AppData\Roaming\Code\User\workspaceStorage\c6617d4ae692e1bb264686cfaa40ad8b\redhat.java\jdt_ws\trabalhojava_73d3de3e\bin' 'cadastroPOO.CadastroPOO'
```

=====

CADASTRO DE CLIENTES

=====

1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo ID
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Sair
Escolha uma opcao: 1

1 - Pessoa Fisica
2 - Pessoa Juridica
Escolha o tipo: 1
ID: 1
Nome: vivian maria
CPF: 123456789
Idade: 26
Pessoa fisica cadastrada.

```
PS C:\Users\vivia\OneDrive\Desktop\trabalhojava> c:: cd 'c:\Users\vivia\OneDrive\Desktop\trabalhojava'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivia\AppData\Roaming\Code\User\workspaceStorage\c6617d4ae692e1bb264686cfaa40ad8b\redhat.java\jdt_ws\trabalhojava_73d3de3e\bin' 'cadastroPOO.CadastroPOO'
```

Pessoa fisica cadastrada.

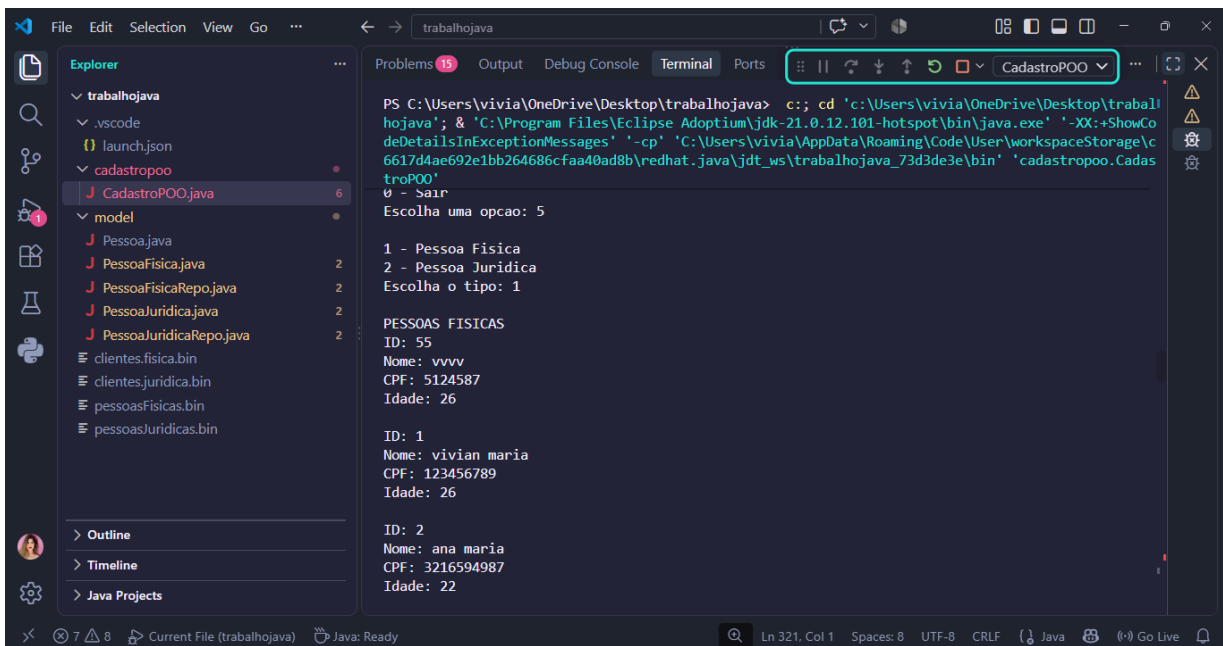
=====

CADASTRO DE CLIENTES

=====

1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo ID
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Sair
Escolha uma opcao: 1

1 - Pessoa Fisica
2 - Pessoa Juridica
Escolha o tipo: 2
ID: 1
Nome: luis mariano
CNPJ: 552147899
Pessoa juridica cadastrada.



```
PS C:\Users\vivia\OneDrive\Desktop\trabalhojava> c:: cd 'c:\Users\vivia\OneDrive\Desktop\trabalhojava'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivia\AppData\Roaming\Code\User\workspaceStorage\c6617d4ae692e1bb264686cfaa40ad8b\redhat.java\jdt_ws\trabalhojava_73d3de3e\bin' 'CadastroPOO.CadastroPOO'
```

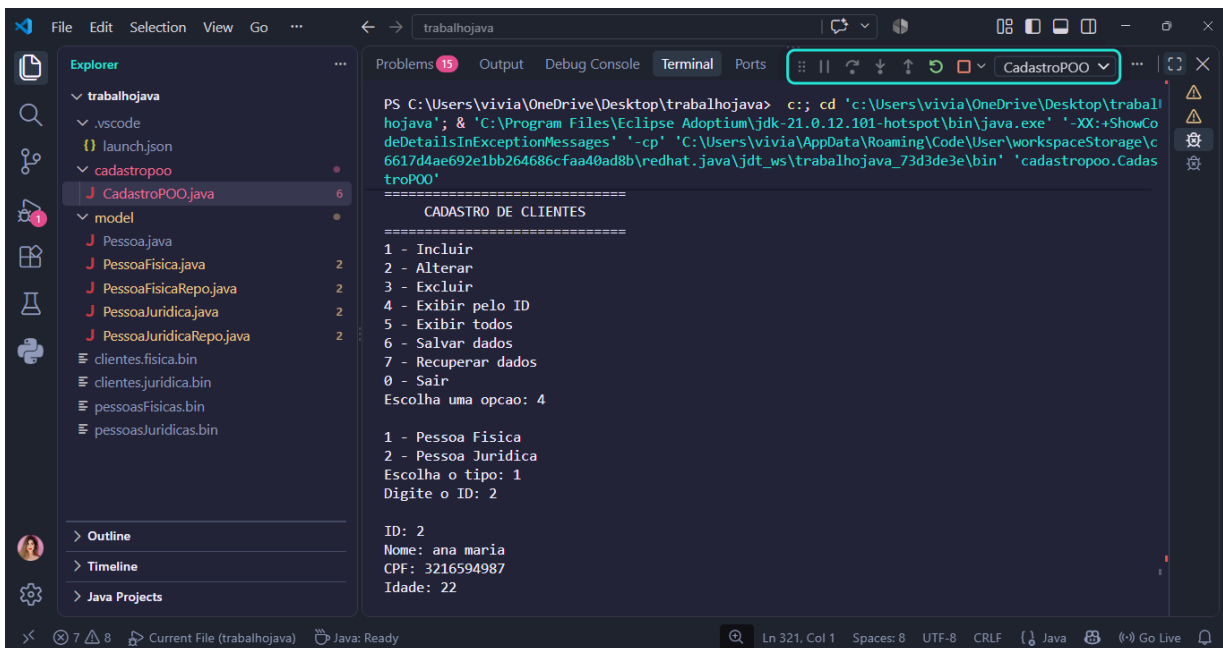
0 - Sair
Escolha uma opcao: 5

1 - Pessoa Fisica
2 - Pessoa Juridica
Escolha o tipo: 1

PESSOAS FISICAS
ID: 55
Nome: vvvv
CPF: 5124587
Idade: 26

ID: 1
Nome: vivian maria
CPF: 123456789
Idade: 26

ID: 2
Nome: ana maria
CPF: 3216594987
Idade: 22



```
PS C:\Users\vivia\OneDrive\Desktop\trabalhojava> c:: cd 'c:\Users\vivia\OneDrive\Desktop\trabalhojava'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivia\AppData\Roaming\Code\User\workspaceStorage\c6617d4ae692e1bb264686cfaa40ad8b\redhat.java\jdt_ws\trabalhojava_73d3de3e\bin' 'CadastroPOO.CadastroPOO'
```

=====

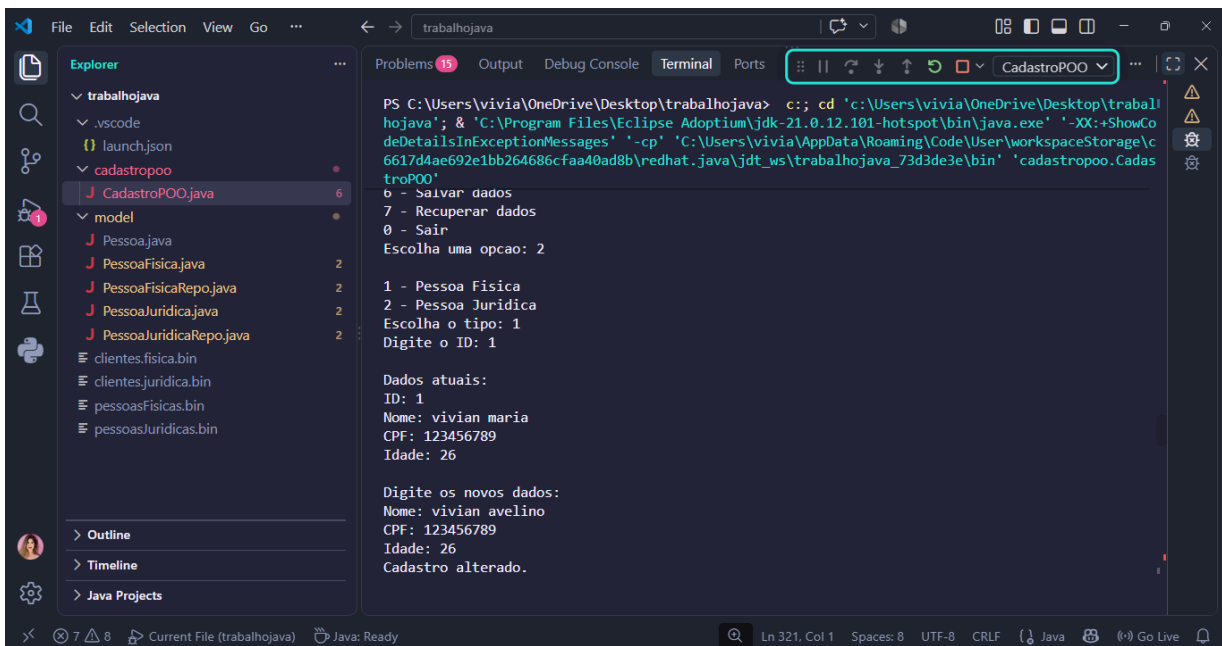
CADASTRO DE CLIENTES

=====

1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo ID
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Sair
Escolha uma opcao: 4

1 - Pessoa Fisica
2 - Pessoa Juridica
Escolha o tipo: 1
Digite o ID: 2

ID: 2
Nome: ana maria
CPF: 3216594987
Idade: 22



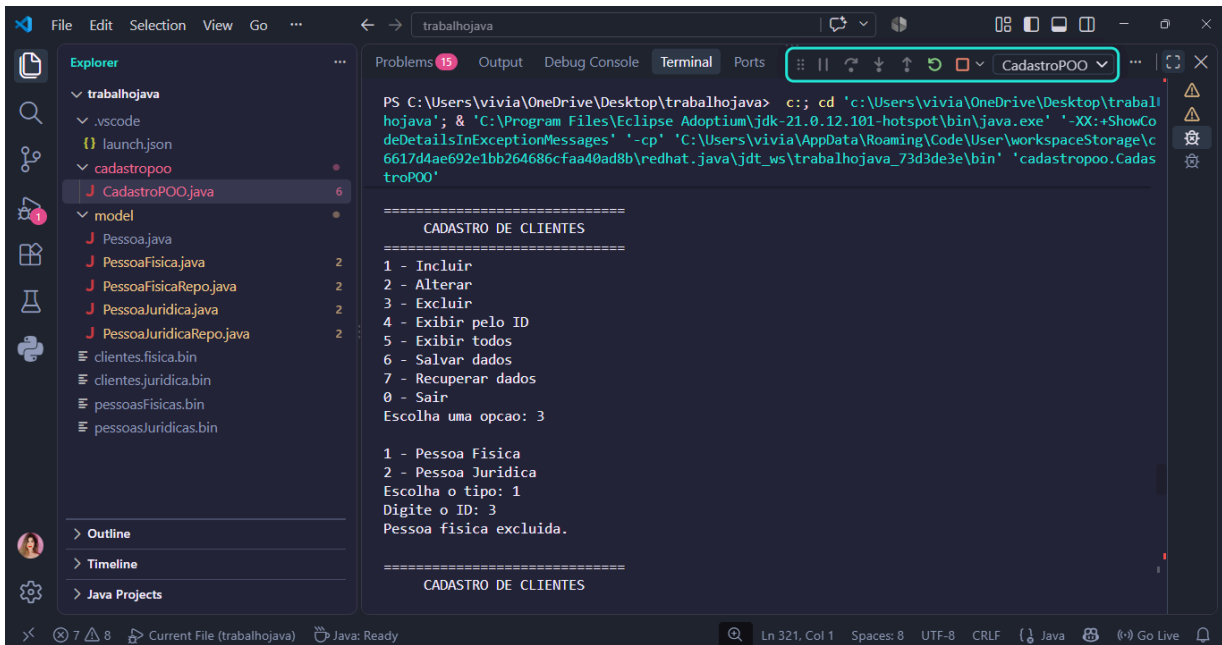
```
PS C:\Users\vivia\OneDrive\Desktop\trabalhojava> c:: cd 'c:\Users\vivia\OneDrive\Desktop\trabalhojava'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivia\AppData\Roaming\Code\User\workspaceStorage\c6617d4ae692e1bb264686cfaa40ad8b\redhat.java\jdt_ws\trabalhojava_73d3de3e\bin' 'CadastroPOO.CadastroPOO'
```

b - Salvar dados
7 - Recuperar dados
0 - Sair
Escolha uma opcao: 2

1 - Pessoa Fisica
2 - Pessoa Juridica
Escolha o tipo: 1
Digite o ID: 1

Dados atuais:
ID: 1
Nome: vivian maria
CPF: 123456789
Idade: 26

Digite os novos dados:
Nome: vivian avelino
CPF: 123456789
Idade: 26
Cadastro alterado.



```
PS C:\Users\vivia\OneDrive\Desktop\trabalhojava> c:: cd 'c:\Users\vivia\OneDrive\Desktop\trabalhojava'; & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vivia\AppData\Roaming\Code\User\workspaceStorage\c6617d4ae692e1bb264686cfaa40ad8b\redhat.java\jdt_ws\trabalhojava_73d3de3e\bin' 'CadastroPOO.CadastroPOO'
```

=====

CADASTRO DE CLIENTES

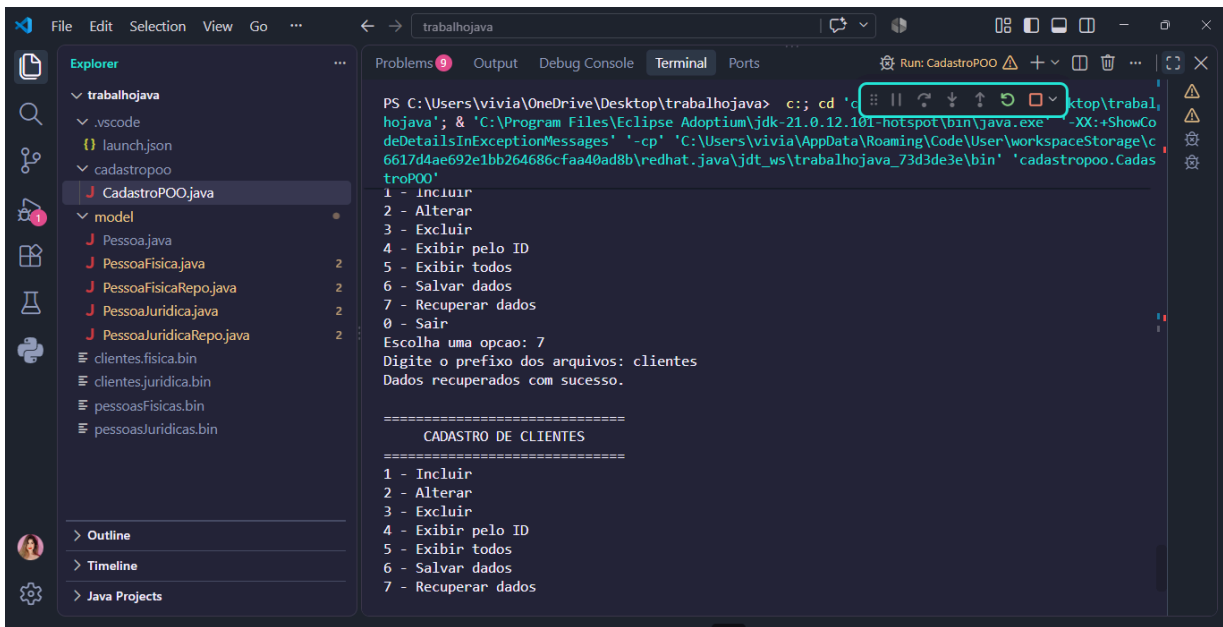
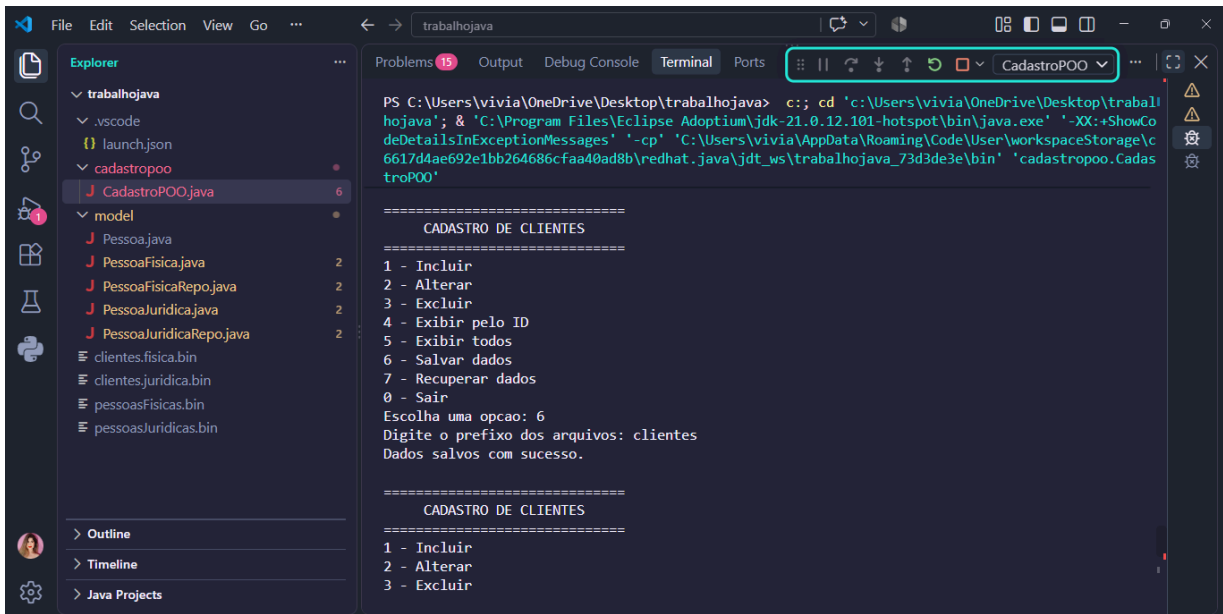
=====

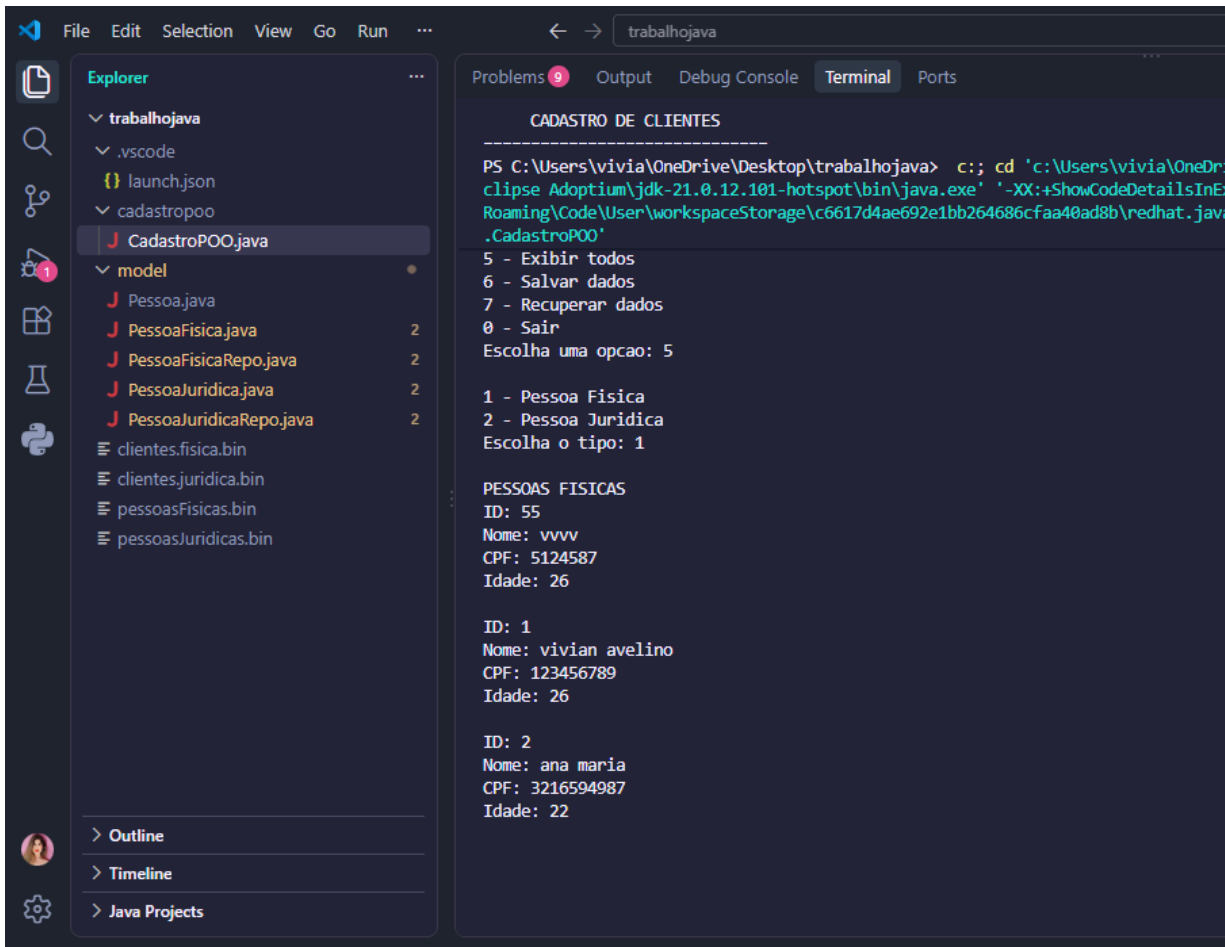
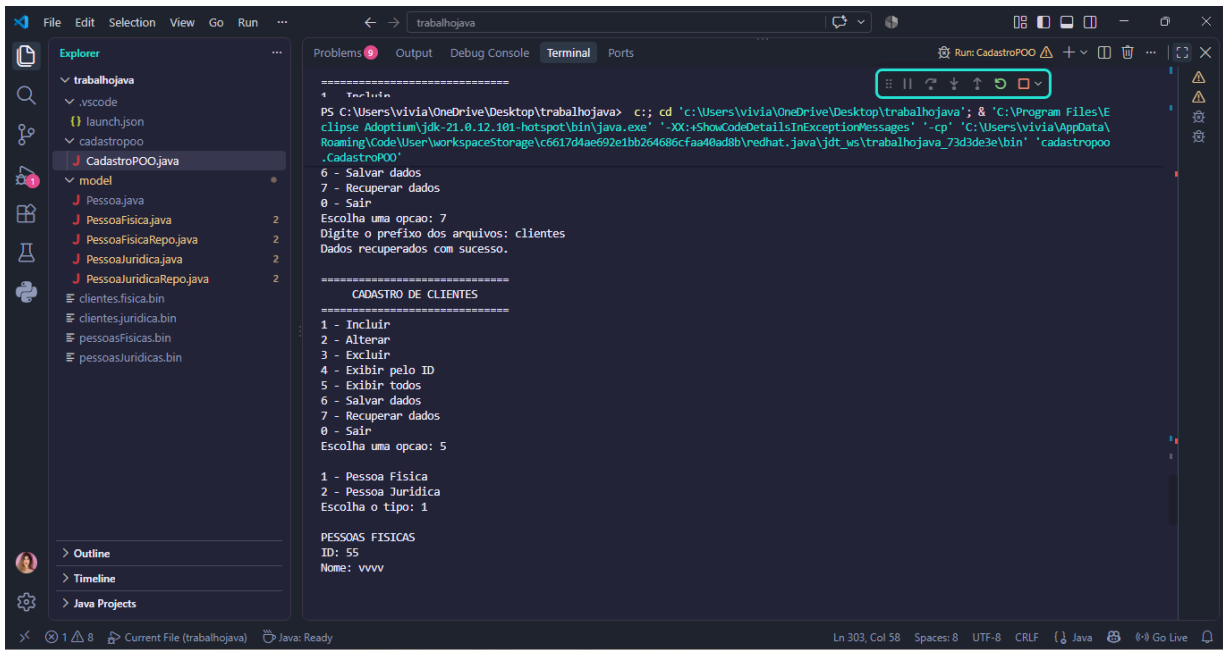
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir pelo ID
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
0 - Sair
Escolha uma opcao: 3

1 - Pessoa Fisica
2 - Pessoa Juridica
Escolha o tipo: 1
Digite o ID: 3
Pessoa fisica excluida.

=====

CADASTRO DE CLIENTES





6. Análise e Conclusão - 1º Procedimento

Quais as vantagens e desvantagens do uso de herança?

A herança permite que uma classe aproveite atributos e métodos de outra classe. Neste projeto, PessoaFisica e PessoaJuridica herdam id e nome da classe Pessoa. Uma vantagem é diminuir a repetição de código e deixar as classes mais organizadas. Uma desvantagem é que as classes filhas ficam ligadas à classe pai, então algumas alterações na classe principal podem afetar as outras.

Por que a interface Serializable é necessária?

A interface Serializable permite que os objetos sejam transformados em uma forma que possa ser gravada em arquivo. Ela foi necessária porque os objetos de PessoaFisica e PessoaJuridica foram armazenados em arquivos binários usando ObjectOutputStream e depois recuperados usando ObjectInputStream.

Como o paradigma funcional é utilizado pela API Stream no Java?

A API Stream permite trabalhar com coleções usando operações como filtrar, transformar e percorrer dados. Ela pode utilizar expressões lambda e ajuda a realizar operações de uma forma mais declarativa. Neste projeto não foi necessário usar Stream, pois as listas foram percorridas com estruturas for simples.

Qual padrão é adotado na persistência de dados em arquivos?

No projeto foram utilizadas classes de repositório para separar as entidades das operações de armazenamento. PessoaFisicaRepo e PessoaJuridicaRepo concentram as operações de inserir, alterar, excluir, consultar, salvar e recuperar. Essa organização segue a ideia do padrão Repository e também possui semelhança com o padrão DAO.

Análise e Conclusão - 2º Procedimento

O que são elementos estáticos e por que main utiliza static?

Elementos estáticos pertencem à própria classe e podem ser utilizados sem criar um objeto daquela classe. O método main utiliza static porque ele é o ponto inicial do programa e precisa ser chamado pela JVM sem que seja necessário criar primeiro um objeto da classe CadastroPOO.

Para que serve a classe Scanner?

A classe Scanner foi utilizada para receber os dados digitados pelo usuário no terminal. Com ela o programa recebe a opção do menu, tipo de pessoa, ID, nome, CPF, CNPJ, idade e o prefixo utilizado para os arquivos.

Como as classes de repositório impactaram na organização do código?

As classes de repositório ajudaram a separar as responsabilidades. As classes Pessoa, PessoaFisica e PessoaJuridica representam os dados dos clientes, enquanto os repositórios cuidam das operações realizadas com esses dados. Isso deixa o programa mais organizado e facilita a localização das funções de cadastro e persistência.

8. Conclusão

Com a realização da prática foi possível aplicar conceitos básicos de programação orientada a objetos em Java. O sistema desenvolvido conseguiu realizar as operações de cadastro solicitadas e também salvar e recuperar os registros em arquivos binários. A atividade ajudou a compreender melhor o uso de herança, polimorfismo, listas, tratamento de exceções e separação do código em classes.

9. Repositório GIT

O projeto deve ser disponibilizado em um repositório GIT. Endereço do repositório:
<https://github.com/vivianavelino/cadastroClientejava.git>